Torc: Durable and Resource-Aware Workflow Orchestration from Workstations to Slurm Clusters

Daniel Thom[1,*], Nathan Keilbart[2]
Joseph McKinsey[1], Pedro Andres Sanchez Perez[1]
Andrew Lai[2], Kapil Duwadi[3]
[1]National Laboratory of the Rockies
[2]Lawrence Livermore National Laboratory
[3]Pacific Northwest National Laboratory
* Corresponding author

15 September 2026

Summary

Computational studies often require researchers to prepare inputs, run many simulations, and combine their results. These tasks may need different computing resources, and failed or changed calculations should not require restarting an entire study. Torc is an open-source workflow manager that coordinates these tasks and retains their execution history. Researchers describe existing programs and their dependencies in a workflow file, then run them on a local computer or a cluster managed by the Slurm scheduler. A command-line interface supports running, inspecting, and selectively repeating work, with graphical and programmatic interfaces for additional access. Torc supports local-first use as well as a persistent shared service, allowing teams to adopt centralized workflow history without administering a separate database service.

Statement of need

Scientific campaigns often begin with shell scripts or scheduler job arrays. These approaches can suffice for independent calculations, but managing dependencies, resource differences, failures, and execution records becomes harder as a study grows. Analysts also need to inspect outputs and trace how they were produced, without becoming developers of a distributed execution system.

Consider a parameter study with input preparation, many independent simulations, and a final aggregation. Preparation and aggregation may use CPUs, while simulations require GPUs or larger memory allocations. A failed simulation should be diagnosed and retried without repeating successful ones; a changed input should invalidate the affected results rather than the entire campaign. Researchers need to manage this lifecycle on a desktop and retain the same workflow model when larger runs require high-performance computing (HPC).

Torc targets this combination of approachable operation, inspectable history, and resource-aware execution. Its intended users include analysts with limited programming experience who run existing scientific applications, and teams that need shared visibility across longer-lived campaigns. Centralized operation is an option, not a prerequisite for starting locally.

State of the field

Scientific workflow systems span several overlapping design spaces. Snakemake and Nextflow provide mature dataflow-oriented workflow languages and broad execution portability (Mölder et al. 2021; Di Tommaso et al. 2017). Pegasus provides planning, provenance, data management, and recovery for large scientific workflows (Deelman et al. 2019). Parsl provides Python-native parallel scripting and high-performance computing executors (Babuji et al. 2019). FireWorks and Balsam provide persistent services and worker-based execution for high-throughput scientific campaigns (Jain et al. 2015; Salim et al. 2018). Pilot systems and hierarchical schedulers, including RADICAL-Pilot and Flux, execute many tasks within larger resource allocations (Merzky et al. 2019; Ahn et al. 2020).

In 2023, Torc's original development team evaluated Nextflow, Snakemake, and Pegasus, and considered FireWorks without deploying it. The team sought a combination suited to analysts running existing applications in large parameter studies: precompiled local deployment without a separate database service, JVM, or conda environment for the orchestrator; declarative command descriptions rather than a workflow programming language; workers that drain a job queue within Slurm allocations; durable, queryable campaign history; integrated recovery; and a language-independent API.

The build-versus-extend decision centered on a persistent server-side state machine and pull-based, resource-aware workers. Rather than having a run-specific driver submit each task through an executor, Torc retains authoritative state in a service from which workers claim suitable jobs. The team judged that combining durable multi-user state with this allocation-level scheduling model would require changes beyond an executor adapter in the evaluated Snakemake and Nextflow designs. FireWorks was a closer architectural fit, but its MongoDB service conflicted with the goal of avoiding separately administered database infrastructure (Jain et al. 2015).

A later team selection exercise favored Torc for local-first use, emphasizing its approachable CLI, binary distribution, optional shared history, and server-worker separation. Snakemake was familiar and positively regarded, but the team preferred Torc's integrated operating model for its intended analysts. This adoption decision is distinct from the 2023 development rationale.

This is a fit-for-purpose rationale, not a claim that other systems cannot execute commands locally or on HPC resources. FireWorks and Balsam are particularly close architectural neighbors in their persistent, worker-based coordination. Torc's contribution is the integration of that operating model with standalone local use, SQLite-backed history, resource-aware execution, and common inspection and recovery tools. The design prioritizes these capabilities within one software package rather than requiring analysts to assemble a workflow engine, a database service, and separate operational tooling. It does not claim to introduce dependency graphs, persistence, or execution inside scheduler allocations.

Software design

Torc separates workflow coordination from execution (Figure 1). An asynchronous Rust server exposes a REST API and stores workflow definitions, dependencies, resource requests, execution results, and events in SQLite. Workflow ownership, project labels, and user-defined metadata provide context for querying retained campaign records. In standalone mode, `torc -s run workflow.yaml` starts a local server for the command's lifetime and retains an on-disk database for later inspection. Alternatively, clients connect to a long-running server for shared history and distributed execution. Precompiled binaries avoid requiring a language build environment on execution hosts; standalone use requires both the CLI and server binaries.

Only the server opens the database; remote workers communicate over HTTP. This permits server-local database storage rather than placing a live database on the cluster's parallel filesystem. SQLite removes the need to administer a separate database service, at the cost of a single-server write bottleneck. Database backups preserve workflow records, while external input files, outputs, and logs must be preserved separately.

A workflow file describes commands, parameter ranges, dependencies, and resource requirements. In the example campaign, parameter expansion creates the simulation jobs, while explicit dependencies or producer-consumer relationships over files and JSON user data connect preparation to simulations and aggregation.

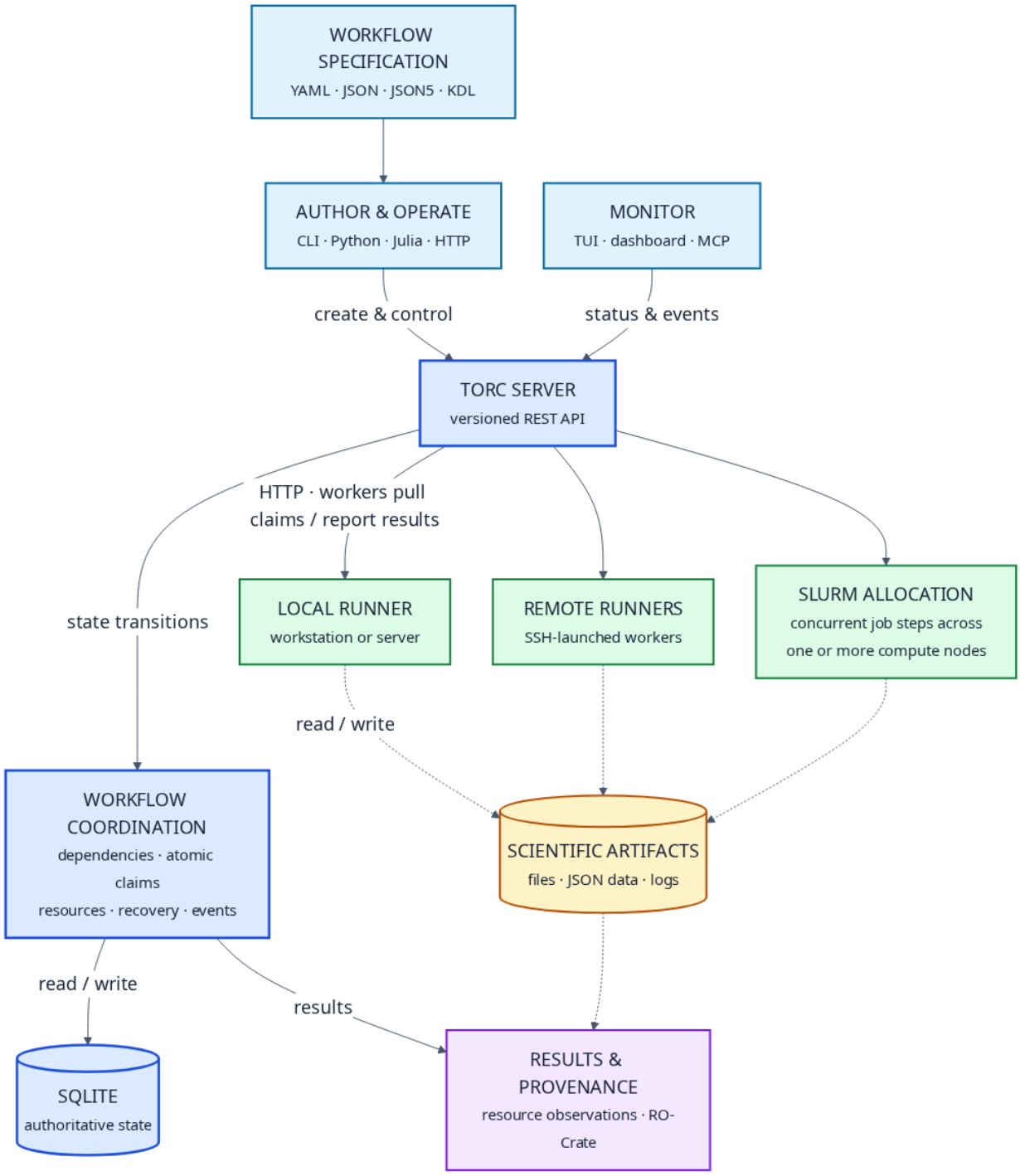


Figure 1: Torc separates user-facing tools, SQLite-backed workflow coordination, and pull-based execution across local, remote, and Slurm resources. Workers claim jobs and report outcomes over HTTP. Solid lines show control and state interactions; dashed lines show scientific artifact access.

These relationships resolve into one dependency graph. Existing scientific executables remain the unit of work rather than requiring their implementation in a workflow-specific programming interface.

Workers request ready jobs that fit their available CPUs, memory, GPUs, node count, and remaining runtime. The server selects jobs and marks them pending in one transaction, preventing competing workers from claiming the same ready job. Priority ordering and residual backfill allow smaller jobs to use capacity left by larger claims. Local and SSH-launched workers execute commands directly; workers inside Slurm allocations can execute directly or launch resource-constrained `srun --exact` job steps. A worker repeatedly claims work within its allocation's remaining walltime, avoiding a separate `sbatch` submission for each application job. Multiple allocations can draw work from the same workflow. Slurm retains allocation and placement responsibilities, while Torc selects dependency-ready work. Moving a workflow to a cluster preserves its dependency model but still requires appropriate application environments, file access, network connectivity, and scheduler configuration.

As jobs finish, recorded outcomes release dependent work and provide an inspectable history of attempts. When monitoring or scheduler accounting is available, CPU, memory, and runtime observations help users diagnose failures and tune subsequent requests. Client-side recovery tools can identify likely memory or time-out failures, adjust resource requests, and retry selected jobs in replacement Slurm allocations. Automatic recovery requires enabling the corresponding monitoring and recovery operation; persistence alone does not perform retries. Workers can journal completions locally during temporary API outages and reconcile them later.

After an input or job definition changes, reinitialization resets affected jobs and their descendants, preserving unaffected branches. Adaptive workflows can also add jobs transactionally during execution, with lineage and iteration limits. Optional RO-Crate generation records workflow, job, input, output, and software provenance (Soiland-Reyes et al. 2022). An OpenAPI contract supports generated Python and Julia clients included in the repository and clients in other languages. The CLI, terminal interface, web dashboard, and Model Context Protocol (MCP) server use the same REST service, keeping inspection and control tied to one workflow state model.

Development in 2025–2026 extended this foundation with MCP-based AI assistance for inspection, debugging, and recovery; expanded dashboard and terminal interfaces; automated Slurm configuration and scheduling; remote worker execution; RO-Crate provenance; ephemeral standalone servers; and dynamic job addition. Control over the API and workflow state model allowed the team to integrate language-model tools alongside conventional interfaces. AI assistance remains optional: neither execution nor deterministic recovery requires it.

Research impact statement

TODO before submission: Confirm which ExaEpi deployment details may be disclosed and what evidence can be shared with editors or reviewers. Obtain an approved reference or supporting record for the Dane PCM work; a research publication may not be available. Remove or qualify claims that cannot be supported, then remove this note.

At the National Energy Research Scientific Computing Center at Lawrence Berkeley National Laboratory, Torc orchestrated ExaEpi parameter sweeps for predicting epidemiological events, including COVID-19. The campaign ran three million simulations in batches of 100,000 on the Perlmutter system. Its Slurm workflow combined GPU-backed main simulations with CPU-backed setup and postprocessing jobs. In the previous workflow, the delay before a worker picked up and started the next calculation could approach the runtime of a short simulation. Torc reduced this between-job dispatch delay, reducing campaign walltime rather than accelerating the simulations themselves. This experience illustrates the importance of keeping workers supplied with ready work when individual calculations are short.

At Lawrence Livermore National Laboratory's Livermore Computing Dane system, Torc ran power-system production-cost modeling (PCM) calculations across varied input scenarios. Each calculation modeled a year of power-system operation; this is the simulated period, not the execution duration. These CPU-only

calculations ran longer than the ExaEpi simulations. Resource monitoring exposed CPU and memory use for debugging and allowed the team to tune resource requests for subsequent simulations. Together, these applications illustrate the value of dispatching short heterogeneous tasks and retaining resource evidence for longer-running analyses.

Availability

Torc is distributed under the BSD 3-Clause license. Source code, installation instructions, workflow examples, contribution guidance, integration tests, and release artifacts are available at https://github.com/NatLabRockies/torc. A diamond-workflow walkthrough in `paper/README.md` provides local execution and retained-result inspection using the existing example and supporting scripts. Automated tests and generated-client consistency checks cover its Rust server and clients. The version reviewed with this paper will be archived with a persistent DOI, and citation metadata will be provided in the repository.

AI usage disclosure

OpenAI GPT-5.6 and GPT-6 Astra Project, accessed through OpenCode, were used for literature discovery, manuscript brainstorming, drafting, revision, figure preparation, and formatting this paper for JOSS. GPT-6 Astra Project also assisted with checking manuscript formatting and citation resolution. Before submission, the authors will review and edit all AI-assisted text and verify its technical claims, references, and comparisons against the software, primary literature, and released evidence. The authors made the software's core design decisions and retain responsibility for the final manuscript.

Acknowledgements

Acknowledgements and funding information will be added before submission.

References

Ahn, Dong H., Nicholas Bass, Albert Chu, Jim Garlick, Mark Grondona, Don Lipari, Becky Martin, Matt Stearman, Becky Springmeyer, and Tapasya Yang. 2020. "Flux: Overcoming Scheduling Challenges for Exascale Workflows." *Future Generation Computer Systems* 110: 202–13. https://doi.org/10.1016/j.future.2020.04.006.

Babuji, Yadu, Anna Woodard, Zhuozhao Li, Daniel S. Katz, Ben Clifford, Rohan Kumar, Lukasz Lacinski, et al. 2019. "Parsl: Pervasive Parallel Programming in Python." In *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, 25–36. New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3307681.3325400.

Deelman, Ewa, Karan Vahi, Mats Rynge, Rajiv Mayani, Rafael Ferreira da Silva, George Papadimitriou, and Miron Livny. 2019. "The Evolution of the Pegasus Workflow Management Software." *Computing in Science & Engineering* 21 (4): 22–36. https://doi.org/10.1109/MCSE.2019.2919690.

Di Tommaso, Paolo, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. 2017. "Nextflow Enables Reproducible Computational Workflows." *Nature Biotechnology* 35 (4): 316–19. https://doi.org/10.1038/nbt.3820.

Jain, Anubhav, Shyue Ping Ong, Wei Chen, Bharat Medasani, Xiaohui Qu, Michael Kocher, Miriam Brafman, et al. 2015. "FireWorks: A Dynamic Workflow System Designed for High-Throughput Applications." *Concurrency and Computation: Practice and Experience* 27 (17): 5037–59. https://doi.org/10.1002/cpe.3505.

Merzky, Andre, Matteo Turilli, Manuel Maldonado, Mark Santcroos, and Shantenu Jha. 2019. "Using Pilot Systems to Execute Many-Task Workloads on Supercomputers." In *Job Scheduling Strategies for Parallel Processing*, 61–82. Lecture Notes in Computer Science. Springer International Publishing. https://doi.org/10.1007/978-3-030-10632-4_4.

Mölder, Felix, Kim Philipp Jablonski, Brice Letcher, Michael B. Hall, Christopher H. Tomkins-Tinch, Vanessa Sochat, Jan Forster, et al. 2021. "Sustainable Data Analysis with Snakemake." *F1000Research* 10: 33. https://doi.org/10.12688/f1000research.29032.2.

Salim, Michael A., Thomas D. Uram, J. Taylor Childers, Prasanna Balaprakash, Venkatram Vishwanath, and Michael E. Papka. 2018. "Balsam: Automated Scheduling and Execution of Dynamic, Data-Intensive HPC Workflows." In *Proceedings of the 8th Workshop on Python for High-Performance and Scientific Computing*. New York, NY, USA: Association for Computing Machinery.

Soiland-Reyes, Stian, Peter Sefton, Mercè Crosas, Leyla Jael Castro, Frederik Coppens, José M. Fernández, Daniel Garijo, et al. 2022. "Packaging Research Artefacts with RO-Crate." *Data Science* 5 (2): 97–138. https://doi.org/10.3233/DS-210053.